

Lesson 08 · Chapter 7: Tích hợp OpenFeign với Service Discovery (Eureka)

• Spring Cloud 2025.x · Spring Boot 3.5.x · Java
21

• ATS Project
Demo

MỤC LỤC

3.1 Painpoints khi thiếu Eureka

3.2 Eureka hoạt động thế nào

3.3 Dựng Eureka Server

3.4 Đăng ký ATS Services

3.5 OpenFeign + Eureka

3.6 Load Balancing

3.7 Luồng end-to-end

3.8 Troubleshooting

PHẦN 3

Tích hợp OpenFeign với Service Discovery (Eureka)

Một trong những tính năng đặc sắc nhất của Spring Cloud OpenFeign chính là khả năng tích hợp mượt mà với Eureka Service Discovery. Thay vì cấu hình URL cứng cho từng service, Feign tự động tra cứu địa chỉ IP:Port từ Eureka Registry tại runtime - giúp hệ thống linh hoạt hơn và không phụ thuộc vào hạ tầng cụ thể. Phần này thực hành trực tiếp trên hệ thống ATS.

3.1 Painpoints khi chưa có Service Discovery

Để thấy rõ giá trị của Eureka, hãy bắt đầu bằng câu hỏi: *điều gì xảy ra nếu dự án ATS không có Service Discovery?*

Trong ATS, các service giao tiếp với nhau rất nhiều: khi candidate nộp CV, **Application Service** cần gọi sang **Candidate Service** để xác thực candidate tồn tại, rồi gọi **Job Service** để kiểm tra job deadline còn hiệu lực, sau đó gọi **Notification Service** để gửi auto-reply email cho candidate. Nếu không có Service Discovery, mỗi lời gọi đó đều phải hardcode URL.

Không có Service Discovery - Vấn đề thực tế

URL cứng trong cấu hình

Mỗi service phải biết địa chỉ vật lý của service khác. Khi deploy lên môi trường khác (dev : staging : prod) phải sửa lại toàn bộ config.

Scale-out thủ công

Nếu `notification-service` cần chạy 3 instance để xử lý tải, cần tự viết load balancer hoặc cấu hình reverse proxy.

Không phát hiện service chết

Nếu `job-service` crash, `application-service` không biết - cứ tiếp tục gọi vào địa chỉ chết cho đến khi timeout.

Không có health check thời runtime

Không có cơ chế tự động loại bỏ instance bị lỗi khỏi danh sách gọi.

Có Eureka Service Discovery - Giải pháp

Gọi bằng tên logical

Chỉ cần biết tên service (`job-service`). Eureka tự tra cứu IP:Port hiện tại. Deploy sang môi trường mới không cần sửa config.

Scale-out tự động

Chạy thêm instance mới của `notification-service`, Eureka tự động đăng ký và Feign sẽ phân phối request đều.

Health check liên tục

Mỗi 30 giây Eureka kiểm tra heartbeat. Instance chết sẽ bị loại khỏi registry sau 90 giây.

Dashboard tập trung

Eureka UI tại `:8761` hiển thị toàn bộ trạng thái hệ thống trong một trang duy nhất.

VẤN ĐỀ CỤ THỂ TRONG ATS KHI KHÔNG CÓ EUREKA

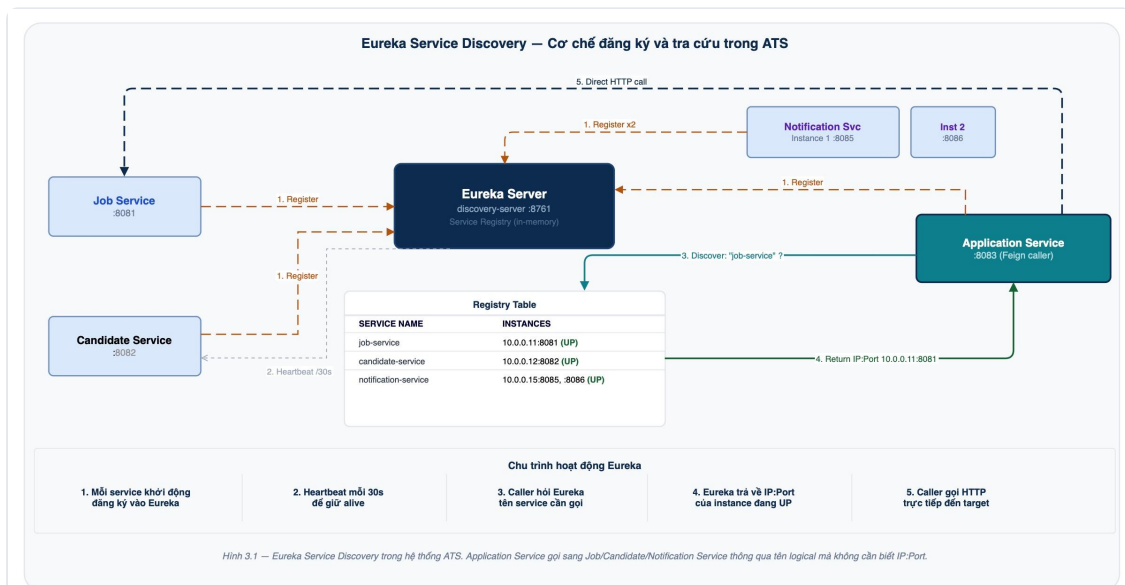
Hãy thử tưởng tượng luồng: candidate nộp đơn vào job tại `application-service`. Service này cần gọi 3 service khác:

1. `job-service:8081` - kiểm tra job còn mở và deadline chưa qua
2. `candidate-service:8082` - xác thực candidate tồn tại
3. `notification-service:8085` - gửi email xác nhận

Nếu `notification-service` được scale lên 2 instance (port 8085 và 8086), `application-service` không thể tự biết instance nào còn sống và cần gọi vào đâu. Với Eureka, toàn bộ việc này được xử lý tự động.

3.2 Eureka hoạt động thế nào

Trước khi đi vào cấu hình, cần hiểu rõ cơ chế hoạt động của Eureka để tránh hiểu lầm khi debug sau này.



Hình 3.1 - Eureka Service Discovery trong hệ thống ATS. Application Service gọi sang Job/Candidate/Notification Service thông qua tên logical mà không cần biết IP:Port.

KHÁI NIỆM	MÔ TẢ	GIÁ TRỊ MẶC ĐỊNH
Service Registration	Khi khởi động, mỗi service tự động đăng ký tên, IP, port và metadata vào Eureka. Tên đăng ký là giá trị của <code>spring.application.name</code> .	Kích hoạt khi có <code>spring-cloud-starter-netflix-eureka-client</code>
Heartbeat	Mỗi service gửi heartbeat định kỳ cho Eureka để chứng minh mình vẫn đang chạy.	Mỗi 30 giây

KHÁI NIỆM	MÔ TẢ	GIÁ TRỊ MẶC ĐỊNH
	Nếu không nhận được heartbeat, Eureka đánh dấu instance là DOWN sau 90 giây.	
Service Discovery	Khi một service cần gọi service khác, nó hỏi Eureka theo tên để lấy danh sách các instance đang UP. Feign của người gọi có local cache (Ribbon cache) để tránh hỏi Eureka mỗi request.	Cache refresh mỗi 30 giây
Self-preservation Mode	Khi Eureka mất liên lạc với nhiều service cùng lúc (ví dụ: network partition), nó sẽ <i>không</i> xóa các instance khỏi registry để tránh sai lầm. Đây là hành vi chủ động bảo toàn registry.	Bật theo mặc định; nên tắt trong môi trường dev
Eureka Client Cache	Mỗi Eureka client lưu local cache về registry để không cần hỏi server mỗi request. Điều này có nghĩa: sau khi service mới đăng ký, có thể mất đến 60–90 giây để các service khác "thấy" nó.	Refresh mỗi 30 giây

3.3 Dụng Eureka Server cho ATS

Trong ATS, Eureka Server chạy như một infrastructure service riêng biệt, tương tự như API Gateway hay Config Server. Tất cả các bước dưới đây thực hiện trong module `eureka-server`.

1 Tạo module eureka-server - Thêm dependency

Tạo một Spring Boot module mới trong project ATS. Module này chỉ cần một dependency duy nhất là `spring-cloud-starter-netflix-eureka-server`.

```
<!-- eureka-server/pom.xml -->
<parent>
```

```

<groupId>com.ats</groupId>

<artifactId>ats-parent</artifactId>

<version>1.0.0-SNAPSHOT</version>
</parent>

<artifactId>eureka-server</artifactId>

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
  </dependency>
  <!-- Actuator cho health check -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>

```

2 Khai báo @EnableEurekaServer trên main class

Annotation `@EnableEurekaServer` kích hoạt Eureka Server trong application context. Đây là annotation duy nhất cần thêm vào code Java.

```

eureka-server/src/main/java/com/ats/discovery/DiscoveryServerApplication.java

package com.ats.discovery;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@EnableEurekaServer

```

```

@EnableEurekaServer // Biến application này thành Eureka Registry
public class DiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }
}

```

3 Cấu hình application.yml cho Eureka Server

Eureka Server cần cấu hình để không tự đăng ký chính nó vào registry (vì nó là registry, không phải client).

```

eureka-server/src/main/resources/application.yml

server:
    port: 8761 # Port chuẩn của Eureka

spring:
    application:
        name: eureka-server

eureka:
    instance:
        hostname: localhost

    client:
        # Quan trọng: Eureka Server không cần đăng ký vào chính nó
        register-with-eureka: false
        fetch-registry: false

server:
    # Tắt self-preservation trong dev - tránh Eureka giữ lại instance lỗi
    enable-self-preservation: false

```

```
# Xóa instance chết nhanh hơn trong dev (mặc định 60s)
eviction-interval-timer-in-ms: 5000

management:

  endpoints:

    web:

      exposure:

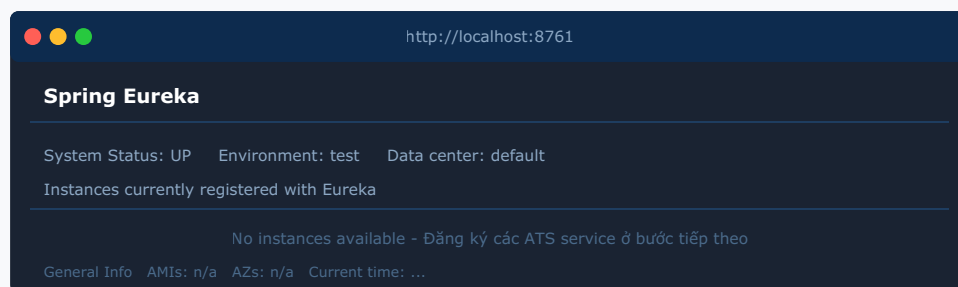
        include: health,info,env
```

ENABLE-SELF-PRESERVATION TRONG PRODUCTION

Trong môi trường production, nên để `enable-self-preservation: true` (mặc định). Chế độ này bảo vệ registry khỏi việc xóa service nhằm khi mạng bị gút do network partition nhất thời. Chỉ tắt trong dev/test để instance chết bị loại nhanh hơn.

4 Xác nhận Eureka Server chạy thành công

Sau khi khởi động, truy cập `http://localhost:8761` để mở Eureka Dashboard. Ban đầu, mục **Instances currently registered with Eureka** sẽ trống - bình thường vì chưa có service nào đăng ký.



Hình 3.2 - Eureka Dashboard tại `localhost:8761` sau khi khởi động. Danh sách instance trống là bình thường.

3.4 Đăng ký các ATS Services vào Eureka

Mọi ATS service cần được cấu hình như một Eureka Client để tự động đăng ký khi khởi động. Cấu hình này giống nhau cho tất cả service - chỉ khác nhau ở `spring.application.name` và port. Dưới đây là hướng dẫn áp dụng cho **Job Service** (làm template cho tất cả service còn lại).

1 Thêm Eureka Client dependency vào mỗi service

```
<!-- job-service/pom.xml - tương tự cho tất cả service -->
<dependencies>

  <!-- Web, JPA, v.v. (đã có từ trước) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Eureka Client - đăng ký service vào registry -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>

  <!-- OpenFeign - gọi sang các service khác -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
  </dependency>
</dependencies>
```

LƯU Ý: EUREKA-CLIENT VÀ OPENFEIGN LÀ HAI DEPENDENCY RIÊNG

`spring-cloud-starter-netflix-eureka-client` giúp service đăng ký vào Eureka. `spring-cloud-starter-openfeign` giúp service gọi

service khác qua Feign. Mỗi service *thường* cần cả hai - vừa đăng ký mình vào registry, vừa có khả năng gọi các service khác.

2 Cấu hình application.yml cho Job Service

```
job-service/src/main/resources/application.yml

server:
  port: 8081

spring:
  application:
    name: job-service  # Tên này sẽ xuất hiện trong Eureka Dashboard
  datasource:
    url: jdbc:postgresql://localhost:5432/ats_job_db
    username: ats_user
    password: ${DB_PASSWORD}

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/  # Địa chỉ Eureka
    fetch-registry: true  # Tải registry về local cache
    register-with-eureka: true  # Đăng ký bản thân vào registry
  instance:
    prefer-ip-address: true  # Dùng IP thay hostname - quan trọng
    lease-renewal-interval-in-seconds: 10  # Heartbeat mỗi 10s (để
    lease-expiration-duration-in-seconds: 30  # Expiry sau 30s không
```

3 Cấu hình các ATS services còn lại

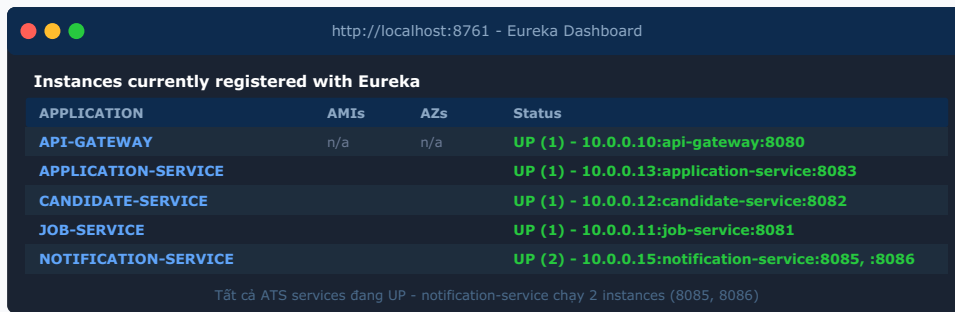
Tất cả các service khác cấu hình tương tự, chỉ thay

`spring.application.name` và `server.port`. Dưới đây là bảng tổng hợp:

SERVICE	SPRING.APPLICATION.NAME	SERVER.PORT	GỢI Ý SERVICE
JOB	job-service	8081	Không service trong A
CANDIDATE	candidate-service	8082	Không service trong A
APPLICATION	application-service	8083	job-s cand: servi noti: servi
NOTIFICATION	notification-service	8085	Không service (lắng n Rabbit
EUREKA	eureka-server	8761	- (là re không chính
GATEWAY	api-gateway	8080	Route service Eureka

4 Xác nhận các service đã đăng ký thành công

Khởi động tất cả các service, sau đó mở Eureka Dashboard tại `http://localhost:8761`. Dashboard sẽ hiển thị tất cả instance đang UP:



The screenshot shows the Eureka Dashboard at http://localhost:8761. It displays a table of instances currently registered with Eureka. The table has four columns: APPLICATION, AMIs, AZs, and Status. The status for each application is 'UP' with a count in parentheses and the instance details.

APPLICATION	AMIs	AZs	Status
API-GATEWAY	n/a	n/a	UP (1) - 10.0.0.10:api-gateway:8080
APPLICATION-SERVICE			UP (1) - 10.0.0.13:application-service:8083
CANDIDATE-SERVICE			UP (1) - 10.0.0.12:candidate-service:8082
JOB-SERVICE			UP (1) - 10.0.0.11:job-service:8081
NOTIFICATION-SERVICE			UP (2) - 10.0.0.15:notification-service:8085, :8086

Tất cả ATS services đang UP - notification-service chạy 2 instances (8085, 8086)

Hình 3.3 - Eureka Dashboard sau khi tất cả ATS services đăng ký thành công.

3.5 OpenFeign tích hợp với Eureka - Demo trong ATS

Đây là phần cốt lõi. Sau khi các service đã đăng ký vào Eureka, việc dùng OpenFeign để gọi giữa các service trở nên rất đơn giản - chỉ cần dùng tên logical (tên đăng ký trong Eureka) thay vì URL cứng.

LUỒNG NỘP ĐƠN ATS - APPLICATION SERVICE GỌI 3 SERVICE

Khi candidate nộp đơn xin việc, `application-service` thực hiện tuần tự:

1. Gọi **job-service** để kiểm tra job còn mở và deadline chưa qua
2. Gọi **candidate-service** để xác thực candidate tồn tại trong hệ thống
3. Gọi **notification-service** để gửi email auto-reply cho candidate

Bước 1 - Khai báo Feign Clients trong Application Service

Tạo package `client` trong `application-service`. Mỗi interface tương ứng với một downstream service. Giá trị `name` trong `@FeignClient` phải khớp chính xác với `spring.application.name` đăng ký trong Eureka.

```
application-service/src/main/java/com/ats/application/client/JobServiceClient.java

package com.ats.application.client;

import com.ats.application.dto.JobDetailResponse;
import org.springframework.cloud.openfeign.FeignClient;
```

```

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

/**
 * Feign client gọi sang job-service qua Eureka.
 * "job-service" phải khớp với spring.application.name của Job Service.
 * Không có url - Feign sẽ resolve từ Eureka Registry.
 */
@FeignClient(
    name      = "job-service",    // Tên đăng ký trong Eureka
    path      = "/api/v1/jobs",  // Base path của Job Service controller
    fallback = JobServiceClientFallback.class
)
public interface JobServiceClient {

    @GetMapping("/{jobId}")
    JobDetailResponse getJobById(@PathVariable("jobId") Long jobId);

    @GetMapping("/{jobId}/is-accepting")
    Boolean isJobAcceptingApplications(@PathVariable("jobId") Long jobId);
}

```

```

application-service/src/main/java/com/ats/application/client/CandidateServiceClient.java

@FeignClient(
    name      = "candidate-service",
    path      = "/api/v1/candidates",
    fallback = CandidateServiceClientFallback.class
)
public interface CandidateServiceClient {

    @GetMapping("/{candidateId}/exists")
    Boolean candidateExists(@PathVariable("candidateId") Long candidateId);

    @GetMapping("/{candidateId}")

```

```

        CandidateResponse getCandidateById(@PathVariable("candidateId") Long candidateId);
    }

```

```

application-service/src/main/java/com/ats/application/client/NotificationServiceClient.java

@FeignClient(
    name      = "notification-service",
    path      = "/api/v1/notifications",
    fallback = NotificationServiceClientFallback.class
)

public interface NotificationServiceClient {

    @PostMapping("/application-received")
    void sendApplicationReceivedEmail(
        @RequestBody ApplicationReceivedEmailRequest request);

    @PostMapping("/pipeline-update")
    void sendPipelineUpdateEmail(
        @RequestBody PipelineUpdateEmailRequest request);
}

```

Bước 2 - Fallback implementations cho tính chịu lỗi

Mỗi Feign client đều cần một fallback class để xử lý trường hợp service đích bị chết hoặc timeout. Fallback trả về giá trị an toàn thay vì để exception bắn ra tới user.

```

application-service/.../client/JobServiceClientFallback.java

@Component

public class JobServiceClientFallback implements JobServiceClient {

    private static final Logger log =
        LoggerFactory.getLogger(JobServiceClientFallback.class);
}

```

```

@Override

public JobDetailResponse getJobById(Long jobId) {

    log.error("[FALLBACK] job-service unavailable, getJobById({})", jobId);

    throw new ServiceUnavailableException(

        "Job service is currently unavailable. Please try again later.");

}

@Override

public Boolean isJobAcceptingApplications(Long jobId) {

    log.error("[FALLBACK] job-service unavailable, isJobAccepting({})", jobId);

    // An toàn: trả về false - không cho apply khi không thể xác nhận

    return false;

}

}

@Component

public class NotificationServiceClientFallback

    implements NotificationServiceClient {

    @Override

    public void sendApplicationReceivedEmail(ApplicationReceivedEmailRequest req) {

        // Email không gửi được - đẩy vào dead letter queue để retry sau

        log.warn("[FALLBACK] notification-service down, email queued for retry: {}",

            req.getCandidateEmail());

    }

}

```

Bước 3 - Sử dụng Feign Clients trong ApplicationService

Đây là nơi toàn bộ luồng nộp đơn ATS được thực thi. Ba Feign client được inject như Spring Bean thông thường.

```

application-service/.../service/ApplicationService.java

```

```

@Service

```

```
@RequiredArgsConstructor

public class ApplicationService {

    private final ApplicationRepository applicationRepository;

    private final JobServiceClient      jobClient;           // Feign proxy
    private final CandidateServiceClient candidateClient;     // Feign proxy
    private final NotificationServiceClient notificationClient; // Feign proxy

    @Transactional

    public ApplicationResponse submitApplication(
        SubmitApplicationRequest request) {

        // 1. Kiểm tra job còn nhận đơn - gọi job-service qua Eureka
        Boolean isAccepting =
            jobClient.isJobAcceptingApplications(request.getJobId());
        if (!isAccepting) {
            throw new JobNotAcceptingException(
                "Job " + request.getJobId() + " is closed or past deadline" );
        }

        // 2. Xác thực candidate tồn tại - gọi candidate-service qua Eureka
        Boolean candidateExists =
            candidateClient.candidateExists(request.getCandidateId());
        if (!candidateExists) {
            throw new CandidateNotFoundException(
                "Candidate " + request.getCandidateId() + " not found");
        }

        // 3. Tạo application record trong DB
        Application application = Application.builder()

            .jobId(request.getJobId())

            .candidateId(request.getCandidateId())

            .status(ApplicationStatus.NEW)

            .appliedAt(LocalDateTime.now())

            .build();
    }
}
```

```

        applicationRepository.save(application);

        // 4. Lấy thông tin candidate để gửi email
        CandidateResponse candidate =
            candidateClient.getCandidateById(request.getCandidateId());
        JobDetailResponse job =
            jobClient.getJobById(request.getJobId());

        // 5. Gửi auto-reply email - gọi notification-service qua Eureka
        notificationClient.sendApplicationReceivedEmail(
            ApplicationReceivedEmailRequest.builder()
                .candidateEmail(candidate.getEmail())
                .candidateName(candidate.getFullName())
                .jobTitle(job.getTitle())
                .applicationId(application.getId())
                .build()
        );

        return ApplicationResponse.from(application);
    }
}

```

Bước 4 - Kích hoạt @EnableFeignClients trong Application Service

```

application-service/.../ApplicationServiceApplication.java

@SpringBootApplication
@EnableFeignClients(basePackages = "com.ats.application.client")
public class ApplicationServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApplicationServiceApplication.class, args);
    }
}

```

Bước 5 - Cấu hình application.yml của Application Service

```
application-service/src/main/resources/application.yml

server:

  port: 8083

spring:
  application:
    name: application-service

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
    fetch-registry: true
    register-with-eureka: true
  instance:
    prefer-ip-address: true

# Timeout per-service - tùy chỉnh theo SLA thực tế
spring:
  cloud:
    openfeign:
      client:
        config:
          default:
            connect-timeout: 2000
            read-timeout: 5000
            logger-level: BASIC
        job-service:
          read-timeout: 3000 # Job query nên nhanh
        notification-service:
          read-timeout: 8000 # Email gửi có thể chậm hơn

# Bật Circuit Breaker cho Feign
```

```

circuitbreaker:
  enabled: true

resilience4j:
  circuitbreaker:
    instances:
      job-service:
        failure-rate-threshold: 50
        wait-duration-in-open-state: 15s
        sliding-window-size: 10
      notification-service:
        failure-rate-threshold: 60 # Notification cho phép fail nhiều hơn
        wait-duration-in-open-state: 30s
        sliding-window-size: 5

logging:
  level:
    com.ats.application.client: DEBUG

```

3.6 Client-side Load Balancing - Scale Notification Service

Một trong những lợi thế của việc kết hợp OpenFeign, Eureka Client và **Spring Cloud LoadBalancer** là khả năng client-side load balancing. Tuy nhiên, LoadBalancer cần dependency và cấu hình `lb://` phù hợp; chỉ có Eureka hoặc OpenFeign thì chưa đủ. Trong ATS, `notification-service` là service có thể bị quá tải nhất do phải gửi email cho nhiều candidate - đây là service cần scale.

Load Balancer là gì?

Load Balancer là thành phần nhận request và chọn một instance trong danh sách các instance của cùng một service. Mục tiêu là phân phối request, tận dụng nhiều instance và tránh gửi request vào instance không còn khả dụng.

THÀNH PHẦN	VAI TRÒ	KHÔNG LÀM VIỆC GÌ?
Eureka Server	Lưu danh sách instance đang đăng ký: tên service, host, port, trạng thái.	Không tự tạo thêm process và không tự scale service.
Eureka Client	Đăng ký instance và tải danh sách instance từ Eureka về local cache.	Không tự chọn instance nếu chưa có LoadBalancer.
Spring Cloud LoadBalancer	Chọn một instance từ danh sách Eureka khi Feign hoặc Gateway gọi service.	Không thay thế Eureka; nó cần service instance list từ discovery.
Docker/ Kubernetes	Chịu trách nhiệm chạy thêm replica khi cần scale.	Không phải thành phần của Spring Cloud LoadBalancer.

ĐIỂM CẦN NHỚ

Eureka không tự scale service. Khi chạy thêm một instance của `candidate-service`, instance mới phải dùng cùng `spring.application.name: candidate-service` nhưng có port hoặc hostname khác. Eureka ghi nhận instance mới, còn Spring Cloud LoadBalancer chọn instance để gửi request.

Spring Cloud LoadBalancer hoạt động như thế nào?

Trong luồng client-side load balancing, service gọi không gọi thẳng vào một URL cố định. Nó gọi bằng tên logical, ví dụ `candidate-service`. LoadBalancer tra local cache, chọn một instance và thay tên service bằng host/port thực tế trước khi gửi HTTP request.

1. Service instance đăng ký vào Eureka.
2. Application Service hoặc API Gateway nhận danh sách instance từ Eureka.
3. Spring Cloud LoadBalancer chọn instance theo strategy, mặc định là Round Robin.
4. Request được gửi tới instance đã chọn.
5. Khi instance dừng heartbeat hoặc bị loại khỏi registry, client sẽ dần ngừng chọn instance đó sau khi cache cập nhật.

LoadBalancer là **client-side**: logic chọn instance nằm trong caller hoặc Gateway. Nó khác với Nginx/HAProxy là **server-side load balancer**, trong đó một proxy trung gian chọn instance.

Thêm dependency cho Eureka và Spring Cloud LoadBalancer

Service nào cần đăng ký vào Eureka thì thêm Eureka Client. Service nào dùng Feign hoặc Gateway để chọn instance qua tên service thì thêm Spring Cloud LoadBalancer.

```
<!-- Thêm vào pom.xml của service cần discovery -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<!-- Cần cho Feign hoặc Gateway dùng lb:// -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

OPENFEIGN KHÔNG DÙNG URL CỐ ĐỊNH

Khi dùng Eureka, không khai báo thuộc tính `url` trong `@FeignClient`. Nếu vẫn dùng `url: http://localhost:8082`, Feign sẽ gọi thẳng một instance và bỏ qua Eureka/LoadBalancer.

```
application-service/src/main/java/fu/ats/client/candidate/
CandidateClient.java

@FeignClient(name = "candidate-service")
public interface CandidateClient {

    @GetMapping("/api/v1/candidates/{candidateId}")
```

```
CandidateView findById(  
    @PathVariable("candidateId") UUID candidateId);  
}
```

Cấu hình Eureka và LoadBalancer cho service gọi

Application Service và các service có Feign client cần biết địa chỉ Eureka, tên logical của mình và timeout. Mỗi instance của cùng service phải giữ nguyên

`spring.application.name` nhưng có `server.port` khác nhau khi chạy local.

```
application-service/src/main/resources/application.yml  
  
server:  
  port: 8083  
  
spring:  
  application:  
    name: application-service  
  cloud:  
    openfeign:  
      client:  
        config:  
          default:  
            connect-timeout: 2000  
            read-timeout: 5000  
  
eureka:  
  client:  
    service-url:  
      defaultZone: http://localhost:8761/eureka/  
    register-with-eureka: true  
    fetch-registry: true  
    registry-fetch-interval-seconds: 5 # Dev: refresh cache nhanh  
    hơn  
  instance:
```

```
instance-id: ${spring.application.name}:${server.port}

prefer-ip-address: true
```

Chỉ cần thay `spring.application.name` và `server.port` cho service khác. Trong production, nên dùng giá trị timeout và refresh phù hợp SLA thay vì giảm quá thấp.

Cấu hình đầy đủ phía API Gateway

API Gateway cũng là Eureka Client. Khi route dùng `lb://service-name`, Gateway lấy danh sách instance từ Eureka và Spring Cloud LoadBalancer chọn instance cho từng request. `uri: http://localhost:8082` chỉ route vào một instance, còn `uri: lb://candidate-service` mới route theo registry.

1 Dependencies trong pom.xml của API Gateway

```
infras/api-gateway/pom.xml

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway-server-
webflux</artifactId>
  </dependency>
  <!-- Kết nối Eureka và đăng ký Gateway -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</
artifactId>
  </dependency>
```

```
<!-- Cho phép uri: lb://... -->

<dependency>

    <groupId>org.springframework.cloud</groupId>

    <artifactId>spring-cloud-starter-loadbalancer</artifactId>

</dependency>

</dependencies>
```

2 Main class

Với Spring Cloud hiện tại, chỉ cần dependency Eureka Client. Không bắt buộc thêm `@EnableDiscoveryClient`; Spring Boot tự cấu hình khi phát hiện starter.

```
infras/api-gateway/src/main/java/fu/ats/ApiGatewayApplication.java

package fu.ats;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication
;

@SpringBootApplication
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class,
args);
    }
}
```

3 application.yml đầy đủ cho API Gateway

```
infras/api-gateway/src/main/resources/application.yml
```

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: job-service
              uri: lb://job-service
              predicates:
                - Path=/api/v1/jobs/**
            - id: candidate-service
              uri: lb://candidate-service
              predicates:
                - Path=/api/v1/candidates/**
            - id: application-service
              uri: lb://application-service
              predicates:
                - Path=/api/v1/applications/**
            - id: notification-service
              uri: lb://notification-service
              predicates:
                - Path=/api/v1/notifications/**
            - id: cv-parse-service
              uri: lb://cv-parse-service
              predicates:
```

```
- Path=/api/v1/cvtparses/**

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
    register-with-eureka: true
    fetch-registry: true
    registry-fetch-interval-seconds: 5
  instance:
    instance-id: ${spring.application.name}:${server.port}
    prefer-ip-address: true

management:
  endpoints:
    web:
      exposure:
        include: health,info
```

SAI LÀM THƯỜNG GẶP

Route dùng `http://localhost:8082` chỉ vào một instance. Để LoadBalancer tham gia, route phải dùng `lb://candidate-service` và Gateway phải có Eureka Client cùng Spring Cloud LoadBalancer.

Nếu Eureka chưa chạy hoặc service chưa đăng ký, Gateway sẽ không tìm thấy instance. Khi đó, kiểm tra Eureka Dashboard trước khi kiểm tra controller.

Scale thủ công và kiểm tra phân phối request

Eureka không tự tạo instance. Khi test local, chạy hai process cùng tên service nhưng khác port:

```
# Terminal 1

mvn -f services/candidate-service/pom.xml spring-boot:run \
-Dspring-boot.run.arguments="--server.port=8082"

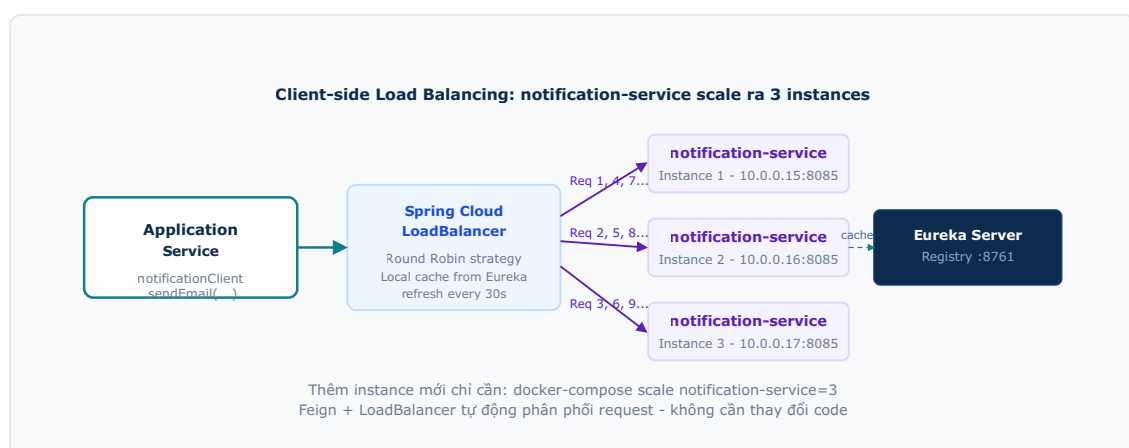
# Terminal 2

mvn -f services/candidate-service/pom.xml spring-boot:run \
-Dspring-boot.run.arguments="--server.port=8086"
```

Cả hai instance phải có `spring.application.name: candidate-service`. Sau khi Eureka nhận đủ hai instance, gửi nhiều request qua Gateway và kiểm tra log của hai instance:

```
# LoggingFilter của Gateway hiện yêu cầu header này
for i in $(seq 1 6); do
    curl -sS -X POST "http://localhost:8080/api/v1/candidates" \
        -H "Content-Type: application/json" \
        -H "X-Correlation-Id: lb-test-$(i)" \
        -d '{"fullName":"Candidate $(i)","email":
        "candidate$(i)@example.com","phone":"0901234$(printf '%03d' $(i))
        ","source":"WEBSITE","skillIds":[]}'
done
```

Kết quả không nhất thiết xen kẽ tuyệt đối, nhưng cả hai instance phải nhận request theo strategy của LoadBalancer.



Hình 3.4 - Khi `notification-service` scale ra 3 instances, `Spring Cloud LoadBalancer` tự động phân phối request theo Round Robin. Không cần thay đổi bất kỳ dòng code nào trong `Application Service`.

THỰC TẾ VỚI DOCKER COMPOSE

Để scale `notification-service` lên 3 instances trong Docker Compose:

```
# Khởi động với 3 replica notification-service
docker-compose up --scale notification-service=3

# Hoặc scale sau khi đã chạy
docker-compose scale notification-service=3
```

Sau khoảng 30–60 giây (thời gian Eureka cache refresh), `application-service` sẽ tự động "thấy" cả 3 instances và bắt đầu phân phối request. Không cần restart hay thay đổi config.

3.7 Luồng end-to-end - Candidate nộp đơn trong ATS

Phần này tổng hợp toàn bộ luồng kỹ thuật từ request của candidate cho đến khi email được gửi, thêm vào các cấu hình Eureka và OpenFeign được sử dụng tại mỗi bước.

- **Candidate gửi POST `/api/v1/applications` (qua API Gateway :8080)**
API Gateway nhận request, tra Eureka để tìm `application-service`, forward request đến instance đang UP tại `10.0.0.13:8083`.
- **`ApplicationService.submitApplication()` được gọi**
Method bắt đầu xử lý. Ba Feign client đã được inject và sẵn sàng.
- **`jobClient.isJobAcceptingApplications(jobId)` được gọi**
Feign proxy tra Eureka local cache, tìm thấy `job-service` tại `10.0.0.11:8081`, gửi HTTP GET `/api/v1/jobs/{jobId}/is-accepting`. Job Service trả về `true`. Thời gian: ~15ms.
- **`candidateClient.candidateExists(candidateId)` được gọi**

Tương tự, Feign gọi `candidate-service` tại `10.0.0.12:8082`. Candidate tồn tại, trả về `true`. Thời gian: ~12ms.

- **Application record được lưu vào database (PostgreSQL `ats_application_db`)**

Status ban đầu: `NEW`. Đây là điểm bắt đầu của pipeline state machine.

- **`notificationClient.sendApplicationReceivedEmail()` được gọi**

Spring Cloud LoadBalancer chọn một trong 3 instances của

`notification-service` (Round Robin). Feign gửi HTTP POST. Notification Service nhận request, đẩy vào RabbitMQ queue để gửi email qua MailHog (dev) hoặc SMTP (prod).

- **Response trả về candidate: `201 Created` với `ApplicationResponse`**

Toàn bộ luồng hoàn thành. Tổng thời gian xử lý (trừ thời gian gửi email): ~50–80ms.

LOG FEIGN TRONG LUỒNG NÀY (LOGGER-LEVEL: BASIC)

```
-- Log từ application-service --
[JobServiceClient]      GET  http://10.0.0.11:8081/api/v1/jobs/42/is-accepting - 2
[CandidateServiceClient] GET  http://10.0.0.12:8082/api/v1/candidates/7/exists - 2
[CandidateServiceClient] GET  http://10.0.0.12:8082/api/v1/candidates/7 - 2
[JobServiceClient]      GET  http://10.0.0.11:8081/api/v1/jobs/42 - 
[NotificationServiceClient] POST http://10.0.0.15:8085/api/v1/notifications/applic
```

3.8 Troubleshooting - Các lỗi thường gặp khi tích hợp Eureka

Phần này tổng hợp những lỗi phổ biến nhất khi kết hợp OpenFeign với Eureka, cùng cách chẩn đoán và sửa lỗi.

TRIỆU CHỨNG / LỖI	NGUYÊN NHÂN	CÁCH KHẮC PHỤC
<code>No instances available for job-service</code>	Feign không tìm thấy instance nào của <code>job-service</code> trong Eureka local cache.	Kiểm tra: (1) <code>job-service</code> đã khởi động chưa, (2) <code>spring.application.name</code> có khớp không, (3) cấu hình <code>eureka.client.service-url.defaultZone</code> có đúng

TRIỆU CHỨNG / LỖI	NGUYÊN NHÂN	CÁCH KHẮC PHỤC
		không. Kiểm tra Eureka Dashboard tại <code>:8761</code> .
Service đăng ký nhưng Feign vẫn báo lỗi 30–60 giây sau khi khởi động	Eureka client cache chưa được refresh sau khi service mới đăng ký. Cache refresh mặc định mỗi 30 giây.	Đây là hành vi bình thường. Trong dev, giảm <code>eureka.client.registry-fetch-interval-seconds: 5</code> để refresh nhanh hơn.
<code>feign.RetryableException: Connection refused</code>	Instance đăng ký trong Eureka nhưng thực tế đã chết (chưa bị loại khỏi registry).	Trong dev: tắt self-preservation và giảm lease expiration. Trong prod: bật Circuit Breaker Resilience4j để tự động chuyển sang instance khác.
<code>Load balancer does not contain an instance for the service</code>	Service chưa đăng ký xong Eureka hoặc <code>prefer-ip-address</code> khiến địa chỉ không thể resolve trong môi trường Docker.	Bật <code>eureka.instance.prefer-ip-address: true</code> trong tất cả service khi chạy với Docker. Kiểm tra <code>eureka.instance.ip-address</code> nếu cần chỉ định IP cụ thể.
Eureka Dashboard hiển thị <code>EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT</code>	Self-preservation mode đang hoạt động - Eureka nghi ngờ mình đang bị mất kết nối với nhiều service.	Trong dev: tắt bằng <code>eureka.server.enable-self-preservation: false</code> . Trong prod: đây là cảnh báo bình thường khi nhiều service khởi động lại cùng lúc.
Một request Feign mất rất lâu (60s+) rồi mới timeout	Feign đang dùng <code>URLConnection</code> mặc định - không có connection pool. Timeout của JVM mặc định rất lâu.	Cấu hình rõ <code>connect-timeout</code> và <code>read-timeout</code> trong YAML. Chuyển sang Apache HC5 hoặc OkHttp để có connection pool.

Cấu hình tối ưu cho dev environment

Trong quá trình phát triển ATS local, có thể thêm các cấu hình sau vào

`application.yml` của tất cả service để Eureka phản hồi nhanh hơn:

```
# Cấu hình Eureka tối ưu cho dev - thêm vào tất cả service
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
    registry-fetch-interval-seconds: 5    # Refresh cache nhanh hơn (mặc định: 30s)
    initial-instance-info-replication-interval-seconds: 5
  instance:
    prefer-ip-address: true
    lease-renewal-interval-in-seconds: 5    # Heartbeat nhanh hơn (mặc định: 30s)
    lease-expiration-duration-in-seconds: 15 # Expiry nhanh hơn (mặc định: 90s)
```

ĐƯỜNG TẮT CẤU HÌNH EUREKA VỚI SPRING PROFILES

Tạo file `application-dev.yml` riêng cho các cấu hình Eureka aggressive như trên. Trong production, dùng giá trị mặc định của Eureka (heartbeat 30s, cache 30s) để giảm tải cho Eureka Server.

TÓM LƯỢC PHẦN 3

Vấn đề gốc: Không có Service Discovery, mỗi inter-service call trong ATS (Application : Job, Candidate, Notification) phải hardcode URL - không scale được, không liệt kê được instance chết, không load balance được tự động.

Giải pháp: Ba thành phần phối hợp: **Eureka Server** (registry lưu danh sách service), **Eureka Client** (mỗi service tự đăng ký và gửi heartbeat), **Spring Cloud LoadBalancer** (Feign dùng để chọn instance khi gọi). OpenFeign đóng vai trò là lớp interface mỏng bên trên, chỉ cần đặt tên service đúng trong

```
@FeignClient(name = "...") .
```

Kết quả trong ATS: `application-service` gọi Job, Candidate, Notification service hoàn toàn thông qua tên logical. Scale `notification-service` lên 3

instances không cần thay đổi một dòng code nào trong caller. Circuit Breaker tự động fallback khi service đích bị chết.

Spring Cloud OpenFeign - Giáo trình kỹ thuật · Phần 3: Tích hợp với Eureka Service Discovery
Demo trên hệ thống ATS (Applicant Tracking System) · Spring Cloud 2025.x · Spring Boot 3.5.x ·
Java 21